Self-Tuning Hyperparameter Optimization via Hamiltonian Dynamics

A Comparative Study of HHD, Hybrid Optimisation, and a Unified Framework

Kartik Desai | CS23B1019 | IIIT Raichur | May 2026

1. Abstract

We present and compare three hyperparameter optimisation strategies for neural networks trained on the harmonic oscillator energy surface $H(q,p)=p^2/(2m)+½kq^2$ and on MNIST CNN classification:

- **(A) Hamiltonian Hyperparameter Dynamics (HHD),** which uses symplectic Hamiltonian Monte Carlo (HMC) [8, 9] to co-evolve network weights and hyperparameters;
- **(B) a Hybrid Adam + L-BFGS + Bayesian Optimisation (ABBO)** method [1, 2, 6]; and
- **(C) a novel Unified HHD-ABBO framework** that integrates all three optimisers into a single three-phase curriculum.

On the harmonic oscillator, the Unified method (Method C) achieves the lowest landscape MSE (0.0033) and highest R^2 (0.9999), outperforming ABBO (Method B) by 30× in error magnitude. On MNIST, Method C achieves 97.70% validation accuracy, completing in 3.3× less wall time than Method B (214.8s vs 698.7s). We provide a comprehensive analysis including dataset visualisation, loss convergence, energy surface reconstruction, Hamiltonian conservation plots, and multi-metric radar comparisons.

2. Hyperparameter Tuning Approach

The central innovation formulates HPO as a physics problem [10, 15, 16]. The training loss $L(\theta, \lambda)$ acts as potential energy V in an extended Hamiltonian where weights (θ) and hyperparameters (λ) are particles with mass and momentum [9, 17]:

$$H(\theta, p\theta, \lambda, p\lambda) = T(p\theta)/m\theta + T(p\lambda)/m\lambda + L(\theta, \lambda)$$

The system evolves via symplectic leapfrog integration [11, 12], ensuring bounded energy error $O(\varepsilon^2)$. Key hyperparameters include learning rate (log-scaled), dropout, depth, width, and batch size - all continuously relaxed [18, 24]. Structural HPs (e.g., layer count) use weight-transfer architecture rebuilding [20]: building a temporary model with the proposed layer count, copying compatible weights, and computing a surrogate gradient.

HP gradients use central finite differences (±10% perturbation); weight gradients use autodifferentiation. An adaptive step-size controller monitors acceptance rate and gradient norm for stability [25, 27]. Metropolis–Hastings correction guarantees detailed balance and ergodicity [9, 10].

3. Proposed Algorithms and Workflow

3.1 Method A - Pure Hamiltonian Hyperparameter Dynamics (HHD)

Method A treats hyperparameter optimization as a purely physical simulation operating in two continuous phases. First, a 20-epoch Adam warm-up [1] descends rapidly into a stable loss basin, preventing chaotic initial dynamics. Second, HMC co-evolution [8, 9] begins: weights and hyperparameters are assigned artificial momenta, and the entire system evolves through the loss landscape using symplectic leapfrog integration over 60 epochs. A Metropolis–Hastings step [10] ensures detailed balance. The hyperparameters glide through smooth, physically interpretable trajectories without discrete restarts. Strength: Guaranteed symplectic energy conservation [11] and elegant physical continuity. Weakness: Lacks second-order curvature information to escape narrow plateaus.

3.2 Method B - Hybrid Adam + L-BFGS + Bayesian Optimization (ABBO)

Method B represents a powerful, decoupled black-box approach that wraps a conventional optimizer sequence inside a probabilistic outer loop. The outer loop uses a Gaussian Process (GP) [7] surrogate model and the Expected Improvement (EI) acquisition function [5, 19] to propose static hyperparameter configurations across 15 independent trials (5 random, 10 GP-guided). Inside each trial, the network is reinitialized and trained from scratch using Adam for 15 epochs, followed by 10 full-batch steps of L-BFGS [2, 4] for tight convergence. Strength: Unmatched reconstruction accuracy due to extensive parallel exploration and L-BFGS precision. Weakness: Extremely computationally expensive due to full network reinitialization and discontinuous hyperparameter jumps [6].

3.3 Method C - Unified HHD-ABBO (Novel)

Method C is the principal novel contribution, integrating the physical elegance of Method A with the second-order precision of Method B into a cohesive three-phase epoch curriculum. There is no outer BO loop; hyperparameters evolve intra-epoch:

- **Phase 1 - Adam Micro-Epochs:** Executes 3 micro-epochs per outer epoch with a cosine-annealed learning rate and gradient clipping [1]. This rapidly navigates the local basin while keeping HPs momentarily frozen.
- **Phase 2 - HMC Co-evolution:** Performs a 6-step leapfrog integration [11] to jointly update θ and λ. An adaptive step-size controller targets a ~65% acceptance rate [10], preserving symplectic energy globally.
- **Phase 3 - Plateau-Triggered L-BFGS:** A continuous monitor [23] checks for local stagnation (tolerance τ=5e-5). If triggered, a strong Wolfe line search [3] executes 30 steps of L-BFGS to exploit second-order curvature [2].

3.4 Execution Workflow

The execution pipeline sequentially runs: config.py → main.py (dynamically instantiating Methods A, B, and C) → results/ serialization → evaluate.py (computing MAE, RMSE, R^2) → generating figures in the plots/ directory.

4. Technologies and Frameworks Used

This project leverages PyTorch [30] for autodifferentiation, dynamic computational graphs, and constructing the neural networks. SciPy [31] handles Gaussian Process surrogate modeling and L-BFGS-B acquisition function maximization for Bayesian Optimization. Custom Python scripts manage the symplectic Hamiltonian integration, Metropolis-Hastings sampling, and the cohesive evaluation pipeline.

5. Validation Methodology

To rigorously quantify theoretical and empirical properties, evaluation routines continuously compute reconstruction residuals and accuracy metrics. The models are tested on two distinct paradigms:

Benchmark	Metric	Calculation & Purpose
Harmonic Osc.	Landscape MAE	Mean Absolute Error: avg(|H_pred - H_true|) across the 6,400-point 2D energy grid.
Harmonic Osc.	Landscape RMSE	Root Mean Squared Error: Highlights severe local deviations in energy prediction.
Harmonic Osc.	R^2 Score	Coefficient of Determination: Quantifies the overall structural fit of the manifold.
MNIST CNN	Val. Accuracy	Percentage of correctly classified unseen images, testing effective regularization.

6. Results

6.1 Harmonic Oscillator Benchmark

Background & Setup: The physics-driven testbed is the Harmonic Oscillator energy surface, utilizing a dense 6,400-point 2D grid. The network aims to reconstruct a quadratic potential $V(q)=½kq^2$. This rigorously tests the algorithm's capability for continuous landscape learning.

Performance: Method C achieved the lowest best validation loss (0.0033 MSE) vs. Method B (0.099) and Method A (0.151). Testbed: Method C - R^2=0.9999 in 81.4s (1.8× faster than B); Method B - R^2=0.9984 in 147.1s. Method A was fastest (17.8s) but least precise (R^2=0.9498). Crucially, only Methods A and C preserved Hamiltonian energy conservation [11, 12], detailed balance [9], and generated smooth continuous HP trajectories.

6.2 CNN MNIST Classification Benchmark

Background & Setup: The secondary testbed is a traditional Convolutional Neural Network (CNN) classification task on the MNIST dataset, explicitly configured with a restricted 5,000 training samples and 1,000 validation samples to purposefully enforce overfitting pressure and demand effective regularization.

Performance: Method A achieved 97.80% validation accuracy, Method C achieved 97.70%, and Method B reached 97.40%. Method C completed in 214.8s - 3.3× less than Method B's 698.7s (which required 15 BO trials). Method A completed in 122.7s. The final dropout selections for Methods A, B, and C converged to 0.21, 0.22, and 0.24 respectively, showing stable regularisation behavior across all models.

6.3 Overall Assessment

Method C consistently balanced accuracy, theoretical guarantees (symplectic conservation [11], detailed balance [9, 10]), second-order convergence [2], and efficiency - validating it as a principled alternative to standalone HMC and traditional BO [6, 13]. While Method B remains the best for pure, unconstrained landscape reconstruction, Method C proves to be the superior framework in realistic, compute-constrained deep learning scenarios.

7. Future Applications of Method C

- **Automated NAS:** Continuous relaxations would let the Hamiltonian system dynamically grow/prune layers in a single physics simulation, eliminating thousands of separate architecture trials [18].
- **LLM Pre-training:** Method C enables on-the-fly self-correction of LR, weight decay, and masking probabilities via energy conservation [11], saving thousands of GPU hours.
- **Physics-Informed Neural Networks:** Addressing gradient pathologies [21, 22, 27] and optimizer selection [23, 28] in PINNs by letting the Hamiltonian system co-evolve PDE-specific hyperparameters adaptively [20, 25].
- **Continual Learning:** HP kinetic energy naturally increases on distribution shift, re-initiating exploration without catastrophic forgetting [26].
- **HPOBench Integration:** Evaluating the Unified HHD-ABBO framework against comprehensive, multi-family hyperparameter optimization benchmark suites like HPOBench [29] to rigorously quantify generalization bounds across diverse dataset modalities. Frameworks like PyTorch [30] and SciPy [31] will enable scaling these implementations.

8. References

[1] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. ICLR, 2015.
[2] D. C. Liu and J. Nocedal, "On the Limited Memory BFGS Method for Large Scale Optimization," Math. Prog., vol. 45, no. 1–3, pp. 503–528, 1989.
[3] P. Wolfe, "Convergence Conditions for Ascent Methods," SIAM Review, vol. 11, no. 2, pp. 226–235, 1969.
[4] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu, "A Limited Memory Algorithm for Bound Constrained Optimization," SIAM J. Sci. Comput., vol. 16, no. 5, pp. 1190–1208, 1995.
[5] J. Mockus, "On Bayesian Methods for Seeking the Extremum," in Optimization Techniques IFIP Technical Conference, 1975.
[6] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of ML Algorithms," in NeurIPS, vol. 25, 2012.
[7] C. E. Rasmussen and C. K. I. Williams, Gaussian Processes for Machine Learning. MIT Press, 2006.
[8] S. Duane, A. D. Kennedy, B. J. Pendleton, and D. Roweth, "Hybrid Monte Carlo," Physics Letters B, vol. 195, no. 2, pp. 216–222, 1987.
[9] R. M. Neal, "MCMC Using Hamiltonian Dynamics," in Handbook of Markov Chain Monte Carlo, CRC Press, 2011, ch. 5, pp. 113–162.
[10] M. Betancourt, "A Conceptual Introduction to Hamiltonian Monte Carlo," arXiv:1701.02434, 2017.
[11] B. Leimkuhler and S. Reich, Simulating Hamiltonian Dynamics. Cambridge University Press, 2004.
[12] E. Hairer, C. Lubich, and G. Wanner, Geometric Numerical Integration, 2nd ed. Springer, 2006.
[13] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," JMLR, vol. 13, pp. 281–305, 2012.
[14] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015.
[15] S. Greydanus, M. Dzamba, and J. Yosinski, "Hamiltonian Neural Networks," in NeurIPS, vol. 32, 2019.
[16] Z. Zhang, S. Bai, Y. Liang, and Z. Bao, "Neural Networks Under Hamiltonian Constraints: A Comprehensive Review," IEEE Access, 2025.
[17] H. Goldstein, C. Poole, and J. Safko, Classical Mechanics, 3rd ed. Addison-Wesley, 2002.
[18] M. A. K. Raiaan et al., "A Systematic Review of HP Optimization Techniques in CNNs," Decision Analytics J., vol. 11, p. 100470, 2024.
[19] P. I. Frazier, "A Tutorial on Bayesian Optimization," arXiv:1807.02811, 2018.
[20] P. Rathore, W. Lei, Z. Frangella, L. Lu, and M. Udell, "Challenges in Training PINNs: A Loss Landscape Perspective," in Proc. 41st ICML, PMLR 235, 2024.
[21] A. Krishnapriyan et al., "Characterizing Possible Failure Modes in Physics-Informed Neural Networks," in NeurIPS, vol. 34, pp. 26548–26560, 2021.
[22] S. Wang, S. Sankaran, H. Wang, and P. Perdikaris, "An Expert's Guide to Training PINNs," arXiv:2308.08468, 2023.
[23] E. Kiyani et al., "Optimizing the Optimizer for PINNs and Kolmogorov-Arnold Networks," arXiv:2501.16371, 2025.
[24] M. Jin et al., "Hyperparameter Tuning of ANNs for Well Production Estimation," ACS Omega, vol. 7, no. 28, pp. 24145–24156, 2022.
[25] W. Chen, A. A. Howard, and P. Stinis, "Self-Adaptive Weights Based on Balanced Residual Decay Rate for PINNs," arXiv:2404.18055, 2024.
[26] L. Newhouse, Mathematical Techniques for Tuning Hyperparameters of RNNs. Research Monograph, 2024.
[27] S. Wang, Y. Teng, and P. Perdikaris, "Understanding and Mitigating Gradient Pathologies in PINNs," SIAM J. Sci. Comput., 2021.
[28] J. C. Wong et al., "Evolutionary Optimization of PINNs: Evo-PINN Frontiers," arXiv:2501.06572, 2025.
[29] K. Eggensperger et al., "HPOBench: A Collection of Reproducible Multi-Fidelity Benchmark Problems for HPO," in NeurIPS Datasets and Benchmarks, 2021.
[30] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in NeurIPS, vol. 32, 2019.
[31] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," Nature Methods, vol. 17, pp. 261–272, 2020.